

Создание шардированной базы с помощью Docker.

Собрал докер-компоуз файлс 4 контейнерами (docker-compose.yml):

config-server - конфиг;

shard1 - шард 1;

shard2 - шард 2;

mongos - маршрутизатор

4 контейнера:

Name	Container ID	Image	Port(s)
▼ ● mongodb-sharding	-	-	-
● config-server	ff863d59344c	mongo:6	27019:27017 ↗
● shard1	c79ffe160aef	mongo:6	27018:27017 ↗
● shard2	ce3ea27d9189	mongo:6	27020:27017 ↗
● mongos	962b59282944	mongo:6	27017:27017 ↗

Создал шарды, конфиг, шардированную коллекцию ratings в базе app_db (поля коллекции user_id, rating, date)

```
test> rs.initiate({
...   _id: "configReplSet",
...   configsvr: true,
...   members: [{ _id: 0, host: "config-server:27017" }]
... })
```

```
test> rs.initiate({
...   _id: "shard1ReplSet",
...   members: [{ _id: 0, host: "shard1:27017" }]
... })
{ ok: 1 }
```

```
test> rs.initiate({
...   _id: "shard2ReplSet",
...   members: [{ _id: 0, host: "shard2:27017" }]
... })
{ ok: 1 }
```

```
[direct: mongos] test> sh.addShard("shard1ReplSet/shard1:27017")
{
  shardAdded: 'shard1ReplSet',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1774029290, i: 6 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1774029290, i: 6 })
}
```

```
[direct: mongos] test> sh.addShard("shard2ReplSet/shard2:27017")
{
  shardAdded: 'shard2ReplSet',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1774029312, i: 5 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1774029312, i: 5 })
}
```

```
[direct: mongos] test> sh.enableSharding("app_db")
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1774029326, i: 6 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1774029326, i: 3 })
}
```

```
[direct: mongos] app_db> db.ratings.createIndex({ "user_id": "hashed" })
user_id_hashed
[direct: mongos] app_db> sh.shardCollection("app_db.ratings", { "user_id": "hashed" })
{
  collectionssharded: 'app_db.ratings',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1774029648, i: 32 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1774029648, i: 28 })
}
```

Заполнил коллекцию для примера:

```
[direct: mongos] app_db> for (let i = 1; i <= 2000; i++) {  
...   db.ratings.insertOne({  
...     user_id: i,  
...     date: "2026-01-" + (Math.floor(Math.random() * 28) + 1),  
...     rating: Math.floor(Math.random() * 5) + 1  
...   })  
... }  
r
```

Получил распределение по шардам:

```
ns: 'app_db.ratings',  
shards: [  
  {  
    shardName: 'shard2ReplSet',  
    numOrphanedDocs: 0,  
    numOwnedDocuments: 1038,  
    ownedSizeBytes: 69546,  
    orphanedSizeBytes: 0  
  },  
  {  
    shardName: 'shard1ReplSet',  
    numOrphanedDocs: 0,  
    numOwnedDocuments: 962,  
    ownedSizeBytes: 64454,  
    orphanedSizeBytes: 0  
  }  
]
```

shard2ReplSet: 51,9%

shard1ReplSet: 48,1%

Консольное приложение на Python (app.py).

1. Добавить запись.

```
=== MongoDB Sharding Demo ===
1. Добавить оценку
2. Подсчет записей по user_id
3. Подсчет записей по rating
4. Общее число записей
5. Выход
Выберите действие: 1
user_id: 1
rating (1-5): 5
date (YYYY-MM-DD): 2026-01-01
Добавлено: user_id=1, rating=5, date=2026-01-01
```

2. Поиск количества записей по заданному user_id (поиск идет лишь в одном шарде, работает быстрее)

```
=== MongoDB Sharding Demo ===
1. Добавить оценку
2. Подсчет записей по user_id
3. Подсчет записей по rating
4. Общее число записей
5. Выход
Выберите действие: 2
user_id: 1
Поиск по user_id=1: найдено записей: 4, время: 0.0030 сек

=== MongoDB Sharding Demo ===
1. Добавить оценку
2. Подсчет записей по user_id
3. Подсчет записей по rating
4. Общее число записей
5. Выход
Выберите действие: 2
user_id: 10
Поиск по user_id=10: найдено записей: 1, время: 0.0028 сек
```

3. Поиск количества записей по заданной оценке rating. Работает дольше, тк не ключевое поле, записи будут в 2 шардах.

```
=== MongoDB Sharding Demo ===
1. Добавить оценку
2. Подсчет записей по user_id
3. Подсчет записей по rating
4. Общее число записей
5. Выход
Выберите действие: 3
rating (1-5): 5
Поиск по rating=5: найдено записей: 368, время: 0.0063 сек
```

```
=== MongoDB Sharding Demo ===
1. Добавить оценку
2. Подсчет записей по user_id
3. Подсчет записей по rating
4. Общее число записей
5. Выход
Выберите действие: 3
rating (1-5): 1
Поиск по rating=1: найдено записей: 405, время: 0.0056 сек
```

4. Подсчет общего числа записей в базе данных.

```
=== MongoDB Sharding Demo ===
1. Добавить оценку
2. Подсчет записей по user_id
3. Подсчет записей по rating
4. Общее число записей
5. Выход
Выберите действие: 4

Всего записей в коллекции: 2003
```

Нагрузочное тестирование (load_test.py).

Здесь параметры запуска для функций прописал хардкодом.

```
=====
НАГРУЗОЧНОЕ ТЕСТИРОВАНИЕ MONGODB SHARDING
=====

Текущее состояние:
  Всего записей: 2100

=== Тест вставки 10000 записей ===
Время: 59.37 сек
Скорость: 168 записей/сек

=== Тест поиска по user_id (500 запросов) ===
Среднее время: 0.0015 сек
Медиана: 0.0015 сек
Мин: 0.0010 сек
Макс: 0.0023 сек

=== Тест поиска по rating (500 запросов) ===
Среднее время: 0.0099 сек
Медиана: 0.0097 сек
Мин: 0.0084 сек
Макс: 0.0142 сек

=====
ТЕСТИРОВАНИЕ ЗАВЕРШЕНО
=====
PS C:\users\PC\projects\mongodb-sharding> |
```

Поиск по ключевому полю, по которому высчитали хэш идет значительно быстрее, так значения лежат в одном шарде, заранее известном.

После нагрузочного тестирования распределение по шардам стало еще равномернее:

```
Shard shard1ReplSet at shard1ReplSet/shard1:27017
{
  data: '398KiB',
  docs: 6033,
  chunks: 2,
  'estimated data per chunk': '199KiB',
  'estimated docs per chunk': 3016
}
---
Shard shard2ReplSet at shard2ReplSet/shard2:27017
{
  data: '400KiB',
  docs: 6067,
  chunks: 2,
  'estimated data per chunk': '200KiB',
  'estimated docs per chunk': 3033
}
---
Totals
{
  data: '798KiB',
  docs: 12100,
  chunks: 4,
  'Shard shard1ReplSet': [
    '49.85 % data',
    '49.85 % docs in cluster',
    '67B avg obj size on shard'
  ],
  'Shard shard2ReplSet': [
    '50.14 % data',
    '50.14 % docs in cluster',
    '67B avg obj size on shard'
  ]
}
[direct: mongos] app_db> |
```